VNU-HCM

University of Science

Faculty of Information Technology

SEMINAR REPORT

Topic: [S06] Diffusion Models: From Adding Noise to Generating Images

Course: CS420 — Introduction to Artificial Intelligence

Students:
Vong Chi Van (24125049)
Le Dat Phuc An (24125052)

November 9, 2026



Contents

1 Introduction **2**

2 Background and Paper Review **3**

2.1 Distinguishing Discriminative vs. Generative Models — Task 1 3

2.2 Synthesis of Core Research Papers 4

3 Method and Step-by-Step Worked Example **5**

3.1 Forward Process, β_t Schedule, Reverse Denoising, and Sampling — Task 2 . . . 5

3.1.1 The Forward Diffusion Process and Variance Preservation 5

3.1.2 Closed-Form t-Step Jump Property 5

3.1.3 Purpose of the β_t Schedule and Schedule Selection Across Data Domains 6

3.1.4 Reverse Denoising Process, Noise Predictor $\boldsymbol{\epsilon}_\theta$, and Why $+\sigma_t \mathbf{z}$ Is Mandatory 6

3.2 Formal Equation Specifications and 1D Worked Numerical Example — Section III & Task 3 7

3.2.1 Step-by-Step 1D Worked Numerical Example (Task 3) 8

3.3 Latent Diffusion Models (LDMs): Perceptual VAE Compression & Cross-Attention — Task 5 10

3.3.1 Perceptual Compression via Frozen Autoencoder $(\mathcal{E}, \mathcal{D})$ and Latent Space Operations 10

3.3.2 Multimodal Conditioning via Cross-Attention 10

4 Results and Discussion **10**

4.1 Part A: Empirical Experiments Developed by Our Team (MNIST 28×28 & 2D Manifolds) 11

4.2 Part B: Pixel-Space Diffusion (DDPM) vs. Latent Diffusion (LDM) — Task 6 . 13

4.3 Critical Analysis and Practical Limitations — Task 6 14

5 Conclusion **15**

Abstract

Diffusion models have emerged as the foundational paradigm of modern Generative AI, surpassing Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), and Autoregressive Models in both visual sample fidelity and mode coverage. This report addresses the central guiding question: *"How can learning to remove noise enable a model to generate brand-new images, and why does latent diffusion drastically reduce computational demands?"* Bridging probabilistic generative modeling with two landmark papers—**Denoising Diffusion Probabilistic Models (DDPM)** by Ho et al. (2020) [1] and **Latent Diffusion Models (LDM)** by Rombach et al. (2022) [2]—we first contrast discriminative classifiers $P_\phi(Y \mid \mathbf{x})$ against generative models $p_\theta(\mathbf{x}, Y) = p_\theta(\mathbf{x} \mid Y)P(Y)$, demonstrating how learned class-conditional Gaussians $\mathcal{N}(\hat{\boldsymbol{\mu}}_c, \hat{\boldsymbol{\Sigma}}_c)$ function both as an optimal Bayes discriminator and as a data generator. We then present a comprehensive analysis of the forward Gaussian noising process $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$, variance-preserving β_t scheduling across data resolutions, the closed-form jump $q(\mathbf{x}_t \mid \mathbf{x}_0)$, the reverse denoising chain $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$, and the simplified noise-prediction objective L_{simple}. Crucially, we prove both intuitively and analytically (via bimodal posterior analysis and the Law of Total Variance) why re-injecting stochastic noise $+\sigma_t \mathbf{z}$ during reverse sampling is strictly necessary to break the symmetry of L_2 regression and prevent trajectories from collapsing onto a blurry dataset mean. We provide complete symbol specifications, a step-by-step 1D numerical worked example, an analysis of perceptual VAE compression combined with Cross-Attention conditioning in latent space, and controlled empirical experiments on 2D manifolds and MNIST (28×28). Finally, we critically evaluate practical limitations regarding sequential sampling latency, prompt-based compositional control, training data copyright/memorization, and societal bias.

Keywords: Generative AI, Denoising Diffusion Probabilistic Models (DDPM), Latent Diffusion Models (LDM), Variational Autoencoder (VAE), Cross-Attention, Bayes' Theorem.

---

1 Introduction

Synthesizing high-dimensional continuous signals such as natural images ($\mathbf{x} \in \mathbb{R}^{H \times W \times C}$) is a central challenge in artificial intelligence. Unlike supervised discriminative learning—which only seeks decision boundaries separating class labels—a generative model must capture the underlying probability distribution $p_{\text{data}}(\mathbf{x})$ of a complex, low-dimensional data manifold embedded in high-dimensional space, enabling the synthesis of novel samples $\mathbf{x}_{\text{new}} \sim p_\theta(\mathbf{x})$.

Prior to diffusion models, deep generative architectures faced the **Generative Learning Trilemma** [9], struggling to simultaneously achieve *(i) high sample sharpness*, *(ii) full mode diversity*, and *(iii) fast sampling*:

1. **Variational Autoencoders (VAEs)** [4] optimize a tractable Evidence Lower Bound (ELBO) and generate images in a single decoder forward pass, but suffer from **blurry samples** because a single-step Gaussian decoder averages over uncertain high-frequency details.
2. **Generative Adversarial Networks (GANs)** [5] achieve razor-sharp, single-step generation via an implicit minimax game ($\min_G \max_D$), but suffer from unstable saddle-point optimization and **mode collapse**—dropping minority modes of the data distribution.

3. **Autoregressive Models (PixelRNN, PixelCNN)** [6] factorize the exact log-likelihood $\prod_{i=1}^{n} p_\theta(x^{(i)} \mid x^{(<i)})$, achieving full mode coverage and sharp local contrast, but require $\mathcal{O}(n)$ sequential forward passes (one per pixel), making high-resolution synthesis prohibitively slow.

Guiding Question and Scope: *How can learning to remove noise enable a model to generate new images, and why does moving diffusion into a learned latent space resolve its computational bottleneck?*

The core insight of **Denoising Diffusion Probabilistic Models (DDPM)** [3, 1] is to decompose the difficult one-step mapping from standard Gaussian noise $\mathcal{N}(\mathbf{0}, \mathbf{I})$ to the data distribution $p_{\text{data}}(\mathbf{x}_0)$ into T small, tractable steps ($T \approx 1000$). If a fixed forward process gradually corrupts a clean image $\mathbf{x}_0$ with small increments of Gaussian noise until $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, then learning the reverse process—repeatedly predicting and subtracting small noise increments ($\mathbf{x}_T \to \mathbf{x}_{T-1} \to \cdots \to \mathbf{x}_0$)—allows a neural network to start from pure random static and iteratively sculpt a realistic image. Furthermore, to overcome the immense computational cost of evaluating a full-resolution U-Net 1000 times in pixel space, **Latent Diffusion Models (LDM)** [2] first compress images into the compact latent space of a pretrained Variational Autoencoder before running diffusion, reducing state dimensionality by 48× and self-attention complexity by 4,096×.

2 Background and Paper Review

2.1 Distinguishing Discriminative vs. Generative Models — Task 1

Supervised classification and probabilistic modeling are fundamentally divided into **Discriminative** and **Generative** paradigms:

1. **Pure Discriminative Models ($P_\phi(Y \mid \mathbf{x})$ — e.g., Logistic Regression, SVMs, MLP/ResNet Classifiers):**
   - **What they learn:** Directly model the conditional probability $P_\phi(Y \mid \mathbf{x})$ or learn a separating hyperplane/decision boundary between classes to maximize classification accuracy, without modeling how the features $\mathbf{x}$ are distributed within each class.
   - **Key Limitations:** Because a discriminative model only knows which side of a decision boundary a point $\mathbf{x}$ falls on, it **cannot generate new samples**. Furthermore, at out-of-distribution (OOD) coordinates far from the data manifold in empty space, a discriminative classifier remains **blindly overconfident ($\approx 100\%$)**, as the query still lies deep on one side of the separating hyperplane.
2. **Generative Models ($p_\theta(\mathbf{x}, Y) = p_\theta(\mathbf{x} \mid Y)P(Y)$ or $p_\theta(\mathbf{x})$ — e.g., Gaussian Bayes / GMM, VAEs, Diffusion Models):**
   - **What they learn:** Explicitly model the underlying data distribution of each class. For instance, in a Gaussian generative model, the model estimates the empirical mean vector $\hat{\boldsymbol{\mu}}_c$ and covariance matrix $\hat{\boldsymbol{\Sigma}}_c$ for each class $c \in \{0, \ldots, C-1\}$:

$$p_\theta(\mathbf{x} \mid Y = c) = \mathcal{N}\big(\mathbf{x};\, \hat{\boldsymbol{\mu}}_c, \hat{\boldsymbol{\Sigma}}_c\big) = \frac{1}{(2\pi)^{D/2}|\hat{\boldsymbol{\Sigma}}_c|^{1/2}} \exp\Big(-\frac{1}{2}(\mathbf{x} - \hat{\boldsymbol{\mu}}_c)^\top \hat{\boldsymbol{\Sigma}}_c^{-1}(\mathbf{x} - \hat{\boldsymbol{\mu}}_c)\Big) \quad (1)$$

- **Using a Learned Generative Model as a Discriminator via Bayes' Theorem:** Once the class distributions $p_\theta(\mathbf{x} \mid Y = c)$ and class priors $P(Y = c)$ are learned, the generative model directly acts as a **classifier (discriminator)** for any test point $\mathbf{x}_q$. It evaluates how likely $\mathbf{x}_q$ is under each class's learned Gaussian distribution and applies **Bayes' Rule**:

$$P_\theta(Y = c \mid \mathbf{x}_q) = \frac{p_\theta(\mathbf{x}_q \mid Y = c)\, P(Y = c)}{\sum_{k=0}^{C-1} p_\theta(\mathbf{x}_q \mid Y = k)\, P(Y = k)}, \qquad \hat{y}_{\text{Bayes}}(\mathbf{x}_q) = \arg\max_c \; p_\theta(\mathbf{x}_q \mid Y = c)\, P(Y = c) \tag{2}$$

  The induced **Bayes decision boundary** between classes c and k is the locus of points where their weighted Gaussian likelihoods are equal: $p_\theta(\mathbf{x} \mid Y = c)P(Y = c) = p_\theta(\mathbf{x} \mid Y = k)P(Y = k)$.

- **Sampling & Out-of-Distribution Detection:** Beyond classification, the marginal data density $p_\theta(\mathbf{x}) = \sum_c P(Y = c)\, p_\theta(\mathbf{x} \mid Y = c)$ enables the model to **detect out-of-distribution anomalies** (where $p_\theta(\mathbf{x}_{\text{OOD}}) \approx 0$) and **synthesize brand-new data samples** $\mathbf{x}_{\text{new}} \sim p_\theta(\mathbf{x})$.

Table 1: Qualitative comparison of major generative model families (Task 1 — The Generative Learning Trilemma).

Model Family	**Density** $p_\theta(\mathbf{x})$	**Training Mechanism & Latent Structure**	**Sample Quality**	**Mode Coverage**	**Sampling Speed**
Autoregressive [6] (PixelRNN, PixelCNN)	Explicit, Exact (*Exact MLE*)	Chain rule $\prod_{i=1}^{n} p_\theta(x^{(i)} \mid x^{(<i)})$ via causal masked convolutions; no global latent $\mathbf{z}$.	Sharp local contrast	Full (No mode collapse)	**Extremely Slow** ($\mathcal{O}(n)$ sequential pixel passes)
VAE [4] (Variational AE)	Explicit, Approx. (ELBO Bound)	Encoder $q_\phi(\mathbf{z} \mid \mathbf{x})$ + Decoder $p_\theta(\mathbf{x} \mid \mathbf{z})$ with $\dim(\mathbf{z}) \ll \dim(\mathbf{x})$; Reparameterization Trick.	**Blurry** (1-step L_2/BCE mode averaging)	Full, smooth latent manifold	**Very Fast** (1 Decoder forward pass)
GAN [5] (DCGAN, StyleGAN)	Implicit (Likelihood-free)	Adversarial minimax game $\min_G \max_D$ between Generator $G_\theta(\mathbf{z})$ and Discriminator $D_\phi(\mathbf{x})$.	**Razor Sharp** (*Mode-seeking*)	**Poor** (Prone to *Mode Collapse* & instability)	**Very Fast** (1 Generator forward pass)
Pixel Diffusion [1] (DDPM)	Explicit, Approx. (T-step ELBO)	Fixed T-step Markov chain with $\dim(\mathbf{x}_t) = \dim(\mathbf{x}_0)$; U-Net predicts noise $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$.	**SOTA Sharpness**	**Full** & stable regression training	**Slow** ($T \approx 1000$ full-res U-Net passes)
Latent Diffusion [2] (LDM / Stable Diff.)	Two-Stage ELBO (VAE + Diffusion)	Compresses $\mathbf{x}_0 \to \mathbf{z}_0$ via frozen Conv-VAE, then runs Cross-Attention U-Net on $\mathbf{z}_t$.	**SOTA** at high resolution	**Full** & flexible multimodal control	**Fast / Practical** (Runs on 48× smaller $\mathbf{z}_t$)

2.2 Synthesis of Core Research Papers

1. **Required Paper — Ho, Jain & Abbeel (NeurIPS 2020): *"Denoising Diffusion Probabilistic Models" (DDPM)* [1]:** Building upon nonequilibrium thermodynamics

[3], Ho et al. established a formal equivalence between hierarchical Markovian variational inference and denoising score matching with Langevin dynamics. Instead of training the reverse network to output the posterior mean image $\tilde{\boldsymbol{\mu}}_t$ directly, the authors reparameterized the U-Net as a **noise predictor** $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ and discarded the theoretical variance-weighting coefficient in the ELBO to obtain L_{simple}. Empirically, Ho et al. demonstrated on CIFAR-10 (32×32) that predicting $\boldsymbol{\epsilon}$ with L_{simple} lowers the Fréchet Inception Distance (FID) from 13.51 to **3.17**—surpassing GANs while preserving full mode diversity.

2. **Recommended Paper — Rombach et al. (CVPR 2022): *"High-Resolution Image Synthesis With Latent Diffusion Models" (LDM)* [2]:** Rombach et al. analyzed the rate-distortion trade-off of pixel-space DDPMs and observed that most model capacity and compute are wasted on imperceptible high-frequency pixel noise. They separated generation into: **(1) Perceptual Compression** via a convolutional autoencoder $(\mathcal{E}, \mathcal{D})$ that down-samples images by factor $f \in \{4, 8\}$ into a spatial latent grid $\mathbf{z}_0 = \mathcal{E}(\mathbf{x}_0)$, and **(2) Semantic Compression** via a diffusion model operating entirely in latent space $\mathbf{z}_t$, augmented with **Cross-Attention** layers for text, layout, and class conditioning.

3 Method and Step-by-Step Worked Example

3.1 Forward Process, β_t Schedule, Reverse Denoising, and Sampling — Task 2

3.1.1 The Forward Diffusion Process and Variance Preservation

Given a clean data sample $\mathbf{x}_0 \sim q(\mathbf{x}_0)$, the **forward process** is a fixed Markov chain (**containing zero learnable parameters**) that gradually injects Gaussian noise over T timesteps according to a variance schedule $\beta_1, \beta_2, \ldots, \beta_T \in (0, 1)$:

$$q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) := \prod_{t=1}^{T} q(\mathbf{x}_t \mid \mathbf{x}_{t-1}), \qquad q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) := \mathcal{N}\Big(\mathbf{x}_t;\, \sqrt{1-\beta_t}\,\mathbf{x}_{t-1},\, \beta_t \mathbf{I}\Big) \tag{3}$$

Via the *Reparameterization Trick*, sampling $\mathbf{x}_t \sim q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$ is expressed as:

$$\mathbf{x}_t = \sqrt{1-\beta_t}\,\mathbf{x}_{t-1} + \sqrt{\beta_t}\,\boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{4}$$

Why scale $\mathbf{x}_{t-1}$ by $\sqrt{1-\beta_t}$? Suppose $\mathrm{Var}(\mathbf{x}_{t-1}) = \mathbf{I}$. Because $\mathbf{x}_{t-1}$ and $\boldsymbol{\epsilon}_t$ are independent, the variance of $\mathbf{x}_t$ is:

$$\mathrm{Var}(\mathbf{x}_t) = \Big(\sqrt{1-\beta_t}\Big)^2 \mathrm{Var}(\mathbf{x}_{t-1}) + \Big(\sqrt{\beta_t}\Big)^2 \mathrm{Var}(\boldsymbol{\epsilon}_t) = (1-\beta_t)\mathbf{I} + \beta_t \mathbf{I} = \mathbf{I} \tag{5}$$

Because the squared coefficients sum to unity ($(1-\beta_t) + \beta_t = 1$), the forward chain is **Variance-Preserving (VP)**, ensuring $\mathbf{x}_T$ converges cleanly to $\mathcal{N}(\mathbf{0}, \mathbf{I})$ without exploding variance.

3.1.2 Closed-Form t-Step Jump Property

Defining $\alpha_t := 1 - \beta_t$ and $\bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s$, and using the additivity of independent Gaussian variances, unrolling the recursion from $\mathbf{x}_0$ to $\mathbf{x}_t$ yields the exact marginal distribution at **any arbitrary timestep t in $\mathcal{O}(1)$ operations**:

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\Big(\mathbf{x}_t;\, \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0,\, (1-\bar{\alpha}_t)\mathbf{I}\Big) \quad \Longleftrightarrow \quad \mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{6}$$

3.1.3 Purpose of the β_t Schedule and Schedule Selection Across Data Domains

If our sole objective in the forward process were to destroy $\mathbf{x}_0$ into $\mathcal{N}(\mathbf{0}, \mathbf{I})$, we could do so in a single step with $\beta_1 = 1$. **The forward β_t schedule exists entirely to lay down "stepping stones" that make the reverse process $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ approximately Gaussian and learnable** (by Feller's Theorem, 1949):

- **Why β_1 is tiny (10^{-4}) while β_T is 200× larger (0.02):** Near $t = 1$, the clean image manifold $q(\mathbf{x}_0)$ is razor-sharp and highly non-Gaussian; a large step would mix multiple distinct modes, making $q(\mathbf{x}_0 \mid \mathbf{x}_1)$ multi-modal. Thus β_1 must be tiny. Near $t = T$, $q(\mathbf{x}_t)$ is already a smooth Gaussian bell curve, allowing much larger steps ($\beta_T = 0.02$).
- **Which β_t schedule to choose for which data:**
  1. *Linear Schedule (Ho et al., 2020 [1], $\beta_1 = 10^{-4} \to \beta_{1000} = 0.02$):* Designed for medium-resolution pixel images (64^2–256^2).
  2. *Cosine Schedule (Nichol & Dhariwal, 2021 [7]):* Defines $\bar{\alpha}_t \propto \cos^2\left(\frac{t/T+s}{1+s}\frac{\pi}{2}\right)$. **Optimal for low-resolution images (28×28 MNIST, 32×32 CIFAR-10)** and low-dimensional data, which lack spatial pixel redundancy and get destroyed too early under a linear schedule.
  3. *Scaled-Linear Schedule (Rombach et al., 2022 [2], $\beta_1 = 0.00085 \to \beta_{1000} = 0.012$ quadratic):* **Optimal for VAE latent tensors ($\mathbf{z}_0 \in \mathbb{R}^{64\times64\times4}$)** where local pixel redundancy has already been compressed out.
  4. *Shifted High-Noise Schedule (Hoogeboom et al., 2023):* **Required for high-resolution images ($512^2, 1024^2$+)**, where 16×16 pixel patches average out independent noise, requiring larger β_t to erase global low-frequency structure.

3.1.4 Reverse Denoising Process, Noise Predictor ϵ_θ, and Why $+\sigma_t\mathbf{z}$ Is Mandatory

The reverse process starts at $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ and learns Gaussian transitions:

$$p_\theta(\mathbf{x}_{0:T}) := p(\mathbf{x}_T)\prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t), \qquad p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) := \mathcal{N}\Big(\mathbf{x}_{t-1};\, \boldsymbol{\mu}_\theta(\mathbf{x}_t, t),\, \sigma_t^2\mathbf{I}\Big) \tag{7}$$

Conditioned on $\mathbf{x}_0$, the forward posterior $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t\mathbf{I})$ is tractable by Bayes' rule:

$$\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t, \qquad \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}\right) \tag{8}$$

Parameterizing the U-Net as a **noise predictor $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$** yields the reverse sampling update for $t = T, \ldots, 1$:

$$\mathbf{x}_{t-1} = \underbrace{\frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right)}_{\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)} + \sigma_t\mathbf{z}, \qquad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \text{ (for } t > 1;\ \mathbf{z} = \mathbf{0} \text{ at } t = 1) \tag{9}$$

Mathematical Proof of Why Omitting $+\sigma_t\mathbf{z}$ Collapses Samples to the Average Image: Because L_{simple} is an L_2 regression loss, the optimal noise predictor outputs the conditional

expectation $\boldsymbol{\epsilon}_\theta^*(\mathbf{x}_t, t) = \mathbb{E}[\boldsymbol{\epsilon} \mid \mathbf{x}_t] = \frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbb{E}[\mathbf{x}_0|\mathbf{x}_t]}{\sqrt{1-\bar{\alpha}_t}}$, which implies:

$$\boldsymbol{\mu}_\theta^*(\mathbf{x}_t, t) = \mathbb{E}[\mathbf{x}_{t-1} \mid \mathbf{x}_t] = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t} \underbrace{\mathbb{E}[\mathbf{x}_0 \mid \mathbf{x}_t]}_{\text{Blurry Posterior Mean}} + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathbf{x}_t \tag{10}$$

Consider a realistic non-Gaussian bimodal dataset with two sharp modes at $x_0 = -1$ (Mode A) and $x_0 = +1$ (Mode B), whose global mean is $\mathbb{E}[x_0] = 0$ (where zero real data exists). By Bayes' rule, the exact posterior expectation is $\mathbb{E}[x_0 \mid x_t] = \tanh\left(\frac{\sqrt{\bar{\alpha}_t}}{1-\bar{\alpha}_t}x_t\right)$.

1. **Without $+\sigma_t z$ ($\sigma_t = 0$):** During early steps ($t \approx T$, where $\bar{\alpha}_t \approx 0$), the mode-splitting term $\tanh(\cdot) \approx 0$ because the Gaussian clouds of Mode A and Mode B completely overlap. Thus, $\boldsymbol{\mu}_\theta^*(x_t, t) \approx \sqrt{\alpha_t}\, x_t$ ($\sqrt{\alpha_t} < 1$), which **actively contracts every trajectory inward toward the blurry mean** $x_t = 0$! By the Law of Total Variance $\mathrm{Var}(x_{t-1}) = \mathrm{Var}(\boldsymbol{\mu}_\theta^*) + \sigma_t^2$, omitting $+\sigma_t z$ strips out σ_t^2 of variance at every step, trapping samples at $x_0 = 0$.
2. **How $+\sigma_t z$ Moves Samples Away from the Mean Into Sharp Modes:** In Stage 1 (large t), $+\sigma_t z$ fights the inward pull and keeps x_t scattered at $\mathcal{O}(1)$ distance away from 0. In Stage 2 (medium/small t), the forward noise clouds around -1 and $+1$ shrink and separate. Because $+\sigma_t z$ moved our sample to, say, $x_{t-1} = +0.5$ (a region visited during training only by Mode B), the L_2 vector field at $x_{t-1} = +0.5$ flips outward: $\tanh\left(\frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}x_{t-1}\right) \to +1$, locking the trajectory onto Mode B ($+1$).

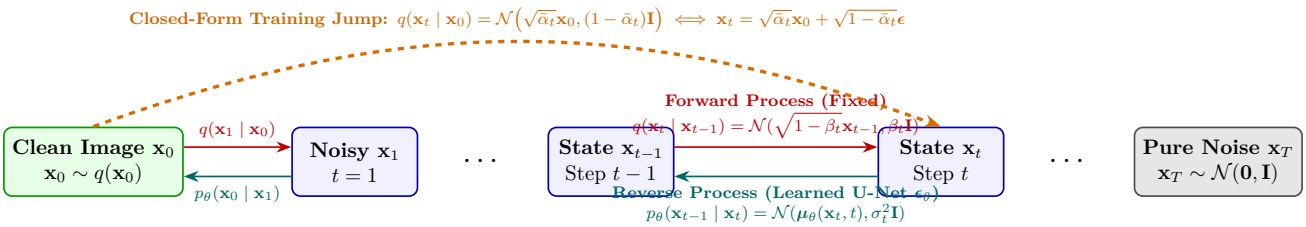


Figure 1: Bidirectional diagram illustrating the forward Gaussian noising process (fixed schedule β_t) and the learned reverse denoising process (driven by U-Net noise predictor $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$) in DDPM (Task 2).

3.2 Formal Equation Specifications and 1D Worked Numerical Example — Section III & Task 3

In accordance with Section III of the seminar specification, Table 2 defines every symbol, unit/value range, and modeling assumption for the two required equations:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}\,\mathbf{x}_{t-1},\ \beta_t\mathbf{I}\right) \tag{Equation 1}$$

$$L_{\text{simple}}(\theta) = \mathbb{E}_{t,\mathbf{x}_0,\boldsymbol{\epsilon}}\left[\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right\|^2\right] \tag{Equation 2}$$

Table 2: Complete symbol definitions, value ranges/units, and modeling assumptions for Section III.

Symbol	**Mathematical Definition**	**Units & Value Range**	**Modeling Assumptions**
t	Discrete diffusion timestep index	Unitless integer: $t \in \{1, 2, \ldots, T\}$ $(T = 1000)$	Finite-horizon discrete-time Markov chain; sampled via $t \sim \text{Uniform}(\{1, \ldots, T\})$ during training.
$\mathbf{x}_0$	Initial clean data sample (image tensor)	Normalized dimensionless pixel intensity: $\mathbf{x}_0 \in [-1, 1]^D$ $(D = H \cdot W \cdot C)$	Continuous state linearly rescaled from $[0, 255]$ integers to $[-1, 1]$ to match the zero-mean, unit-variance prior $\mathcal{N}(\mathbf{0}, \mathbf{I})$.
$\mathbf{x}_{t-1}, \mathbf{x}_t$	Noisy latent states at steps $t-1$ and t	Continuous real tensors: $\mathbf{x}_{t-1}, \mathbf{x}_t \in \mathbb{R}^D$	**Dimension-preserving latent space:** Every latent $\mathbf{x}_t$ has the exact same dimensionality D as $\mathbf{x}_0$.
$q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$	Forward Markov transition probability density	Probability density: $q \geq 0$, $\int q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) d\mathbf{x}_t = 1$	**Markov Property:** $q(\mathbf{x}_t \mid \mathbf{x}_{0:t-1}) = q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$ (memoryless conditioned on $\mathbf{x}_{t-1}$).
β_t	Forward noise variance schedule at step t	Unitless variance fraction: $\beta_t \in (0, 1)$, typically $10^{-4} \leq \beta_t \leq 0.02$	Fixed hyperparameter schedule ($\beta_1 < \cdots < \beta_T$) chosen small enough ($\beta_t \ll 1$) that reverse transitions are Gaussian.
$\sqrt{1-\beta_t}$	Signal attenuation (contraction) factor	Unitless scalar: $\sqrt{1-\beta_t} \in (0, 1)$	Variance-preserving scaling: $(\sqrt{1-\beta_t})^2 + (\sqrt{\beta_t})^2 = 1$.
$\mathbf{I}$	$D \times D$ Identity covariance matrix	Dimensionless matrix $\mathbf{I} \in \{0, 1\}^{D \times D}$	**Isotropic Gaussian assumption:** Noise is independent and identically distributed (i.i.d.) across all pixels/channels.
L_{simple}	Simplified unweighted noise MSE objective	Non-negative real scalar: $L_{\text{simple}} \in [0, \infty)$	Drops the ELBO weight $\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1-\bar{\alpha}_t)}$ to upweight harder, high-noise semantic steps ($t \gg 1$).
$\mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}}[\cdot]$	Joint expectation over $(t, \mathbf{x}_0, \boldsymbol{\epsilon})$	Approximated via mini-batch average of size B	Assumes t, $\mathbf{x}_0 \sim q(\mathbf{x}_0)$, and $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ are drawn independently.
$\boldsymbol{\epsilon}$	Ground-truth standard normal noise realization	Continuous real tensor: $\boldsymbol{\epsilon} \in \mathbb{R}^D$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$	Zero mean, unit variance ($\approx 99.7\%$ of values in $[-3, +3]$).
$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$	Neural network (U-Net) noise predictor	Continuous real tensor: $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \in \mathbb{R}^D$	Shares weights θ across all T steps via sinusoidal timestep embeddings.

3.2.1 Step-by-Step 1D Worked Numerical Example (Task 3)

Consider a 1-dimensional forward noising step ($D = 1$, representing a single normalized pixel) at $t = 1$:

1. **Chosen Input Values:**

   - Clean normalized pixel intensity: $\mathbf{x_0} = \mathbf{0.80}$ (in $[-1, 1]$, corresponding to $229.5 \approx 230$ in 8-bit grayscale).

- Variance schedule at step $t = 1$: $\beta_1 = \mathbf{0.19}$ (chosen so $1-\beta_1 = 0.81$ has an exact rational square root $\sqrt{0.81} = 0.90$; representative of a 5-step illustrative schedule).
- Realized standard normal noise sample drawn from $\epsilon \sim \mathcal{N}(0,1)$: $\epsilon = \mathbf{-0.50}$.

2. **Step 1 — Compute Parameters of the Forward Distribution $q(x_1 \mid x_0 = 0.80)$:**

   - Signal retention factor: $\alpha_1 = 1 - \beta_1 = 1 - 0.19 = 0.81 \implies \sqrt{1-\beta_1} = \sqrt{0.81} = \mathbf{0.90}$.
   - **Stochastic Expectation (Conditional Mean):**

   $$\mu_1 = \mathbb{E}[x_1 \mid x_0 = 0.80] = \sqrt{1-\beta_1}\, x_0 = 0.90 \times 0.80 = \mathbf{0.7200} \tag{11}$$

   - **Conditional Variance and Standard Deviation:**

   $$\sigma_1^2 = \beta_1 = \mathbf{0.1900} \implies \sigma_1 = \sqrt{\beta_1} = \sqrt{0.19} \approx \mathbf{0.4359} \tag{12}$$

   Hence, $q(x_1 \mid x_0 = 0.80) = \mathcal{N}(0.7200,\, 0.1900)$.

3. **Step 2 — Calculate the Realized Noisy Pixel x_1 for $\epsilon = -0.50$:**

$$x_1 = \sqrt{1-\beta_1}\, x_0 + \sqrt{\beta_1}\, \epsilon = 0.7200 + (0.4359 \times (-0.50)) = 0.7200 - 0.21795 = \mathbf{0.5021} \tag{13}$$

4. **Step 3 — Physical Interpretation of the Stochastic Expectation $\mathbb{E}[x_1 \mid x_0] = 0.7200$:** Since $\mathbb{E}[\epsilon] = 0$, averaging over infinitely many random realizations of $\epsilon \sim \mathcal{N}(0,1)$ yields:

$$\mathbb{E}_{\epsilon\sim\mathcal{N}(0,1)}[x_1 \mid x_0] = \sqrt{1-\beta_1}\, x_0 + \sqrt{\beta_1}\, \underbrace{\mathbb{E}[\epsilon]}_{=0} = 0.90\, x_0 = 0.7200 < x_0 \tag{14}$$

   Interpretation: Even when random noise fluctuations average out, the forward diffusion step **systematically contracts the expected signal amplitude toward** 0 by factor $\sqrt{1-\beta_t} = 0.90$. Over T steps, $\mathbb{E}[x_T \mid x_0] = \sqrt{\bar{\alpha}_T}\, x_0 \to 0$, erasing each image's initial mean so all inputs merge into the zero-mean prior $\mathcal{N}(0,1)$.

5. **Step 4 — Worked Calculation of L_{simple} and Reverse Denoising ($x_1 \to \hat{x}_0$):** Suppose the neural network receives ($x_1 = 0.5021$, $t = 1$) and outputs predicted noise $\epsilon_\theta(0.5021, 1) = \mathbf{-0.30}$.

   - **Sample Loss L_{simple}:**

   $$\ell_{\text{simple}} = \left\|\epsilon - \epsilon_\theta(x_1, 1)\right\|^2 = (-0.50 - (-0.30))^2 = (-0.20)^2 = \mathbf{0.0400} \tag{15}$$

   - **Reverse Denoising Step (9) at $t = 1$ (with $z = 0$, $\alpha_1 = \bar{\alpha}_1 = 0.81$):**

   $$\hat{x}_0 = \frac{1}{\sqrt{\alpha_1}}\left(x_1 - \frac{\beta_1}{\sqrt{1-\bar{\alpha}_1}}\epsilon_\theta(x_1, 1)\right) = \frac{1}{0.90}\Big(0.5021 - 0.4359 \times (-0.30)\Big) = \frac{0.6329}{0.90} \approx \mathbf{0.7032} \tag{16}$$

   Once trained to optimality ($\epsilon_\theta = \epsilon = -0.50$), we recover $\hat{x}_0 = \frac{0.5021 - 0.4359(-0.50)}{0.90} = \frac{0.7200}{0.90} = \mathbf{0.8000} = \mathbf{x_0}$.

3.3 Latent Diffusion Models (LDMs): Perceptual VAE Compression & Cross-Attention — Task 5

3.3.1 Perceptual Compression via Frozen Autoencoder $(\mathcal{E}, \mathcal{D})$ and Latent Space Operations

In pixel-space DDPMs, $\mathbf{x}_t$ retains the full resolution of $\mathbf{x}_0$ (e.g., $512 \times 512 \times 3 = 786{,}432$ dimensions), making 1000 U-Net evaluations computationally prohibitive. Rombach et al. (2022) [2] overcome this via a two-stage pipeline (Figure 2):

1. **Perceptual Convolutional VAE $(\mathcal{E}, \mathcal{D})$:** A Fully Convolutional VAE is pretrained with a perceptual loss (LPIPS), a patch-based adversarial discriminator, and a light KL penalty (*KL-reg*) to map $\mathbf{x}_0 \in \mathbb{R}^{H \times W \times 3}$ into a 3D spatial latent tensor $\mathbf{z}_0 = \mathcal{E}(\mathbf{x}_0) \in \mathbb{R}^{h \times w \times c}$ downsampled by factor $f = H/h = W/w = 2^m$.
   - In Stable Diffusion $(f = 8, c = 4)$, a $512 \times 512 \times 3$ image (786,432 scalars) is compressed into $\mathbf{z}_0 \in \mathbb{R}^{64 \times 64 \times 4}$ (16,384 scalars)—a $48\times$ **reduction in tensor size**!
   - Because 2D self-attention scales quadratically with spatial area $\mathcal{O}((H \cdot W)^2)$, reducing spatial dimensions by $f = 8$ reduces token count by $f^2 = 64\times$ and **cuts self-attention memory and FLOPs by** $f^4 = 8^4 = 4{,}096\times$!
2. **Diffusion in Latent Space $(\mathbf{z}_0 \leftrightarrow \mathbf{z}_T)$:** With $\mathcal{E}$ and $\mathcal{D}$ frozen, forward noising $\mathbf{z}_t = \sqrt{\bar{\alpha}_t}\mathbf{z}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}$ and reverse U-Net denoising $\boldsymbol{\epsilon}_\theta(\mathbf{z}_t, t)$ run entirely on $\mathbf{z}_t \in \mathbb{R}^{64 \times 64 \times 4}$. At the end of the T-step reverse loop, a **single forward pass** through the frozen decoder $\hat{\mathbf{x}}_0 = \mathcal{D}(\hat{\mathbf{z}}_0)$ reconstructs the full $512 \times 512 \times 3$ RGB image.

3.3.2 Multimodal Conditioning via Cross-Attention

To condition generation on text prompts or semantic maps y, a domain encoder $\tau_\theta(y) \in \mathbb{R}^{M \times d_\tau}$ (e.g., a pretrained CLIP text transformer) projects y into M token embeddings, which are injected into intermediate U-Net layers via **Multi-Head Cross-Attention**:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}}\right)\mathbf{V}, \quad \begin{cases} \mathbf{Q} = \varphi_i(\mathbf{z}_t)\,\mathbf{W}_Q^{(i)} & \text{(from spatial latent } \mathbf{z}_t\text{)} \\ \mathbf{K} = \tau_\theta(y)\,\mathbf{W}_K^{(i)} & \text{(from prompt } y\text{)} \\ \mathbf{V} = \tau_\theta(y)\,\mathbf{W}_V^{(i)} & \text{(from prompt } y\text{)} \end{cases} \tag{17}$$

The conditioned Latent Diffusion training objective is:

$$L_{\text{LDM}}(\theta) := \mathbb{E}_{\mathcal{E}(\mathbf{x}_0),\, y,\, \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),\, t}\left[\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\big(\mathbf{z}_t,\, t,\, \tau_\theta(y)\big)\right\|^2\right] \tag{18}$$

4 Results and Discussion

In strict compliance with the seminar guidelines (distinguishing experiments developed by our team from findings reported in the research literature), we divide our results into two parts.

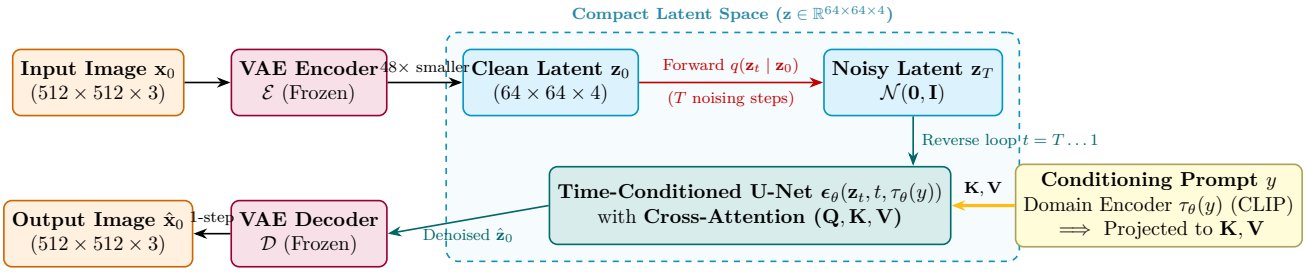


Figure 2: Complete Latent Diffusion Model (LDM) pipeline: Perceptual compression via frozen VAE $(\mathcal{E}, \mathcal{D})$, diffusion operations in latent space $\mathbf{z}$, and Cross-Attention conditioning (Task 5, adapted from Rombach et al., 2022 [2]).

4.1 Part A: Empirical Experiments Developed by Our Team (MNIST 28×28 & 2D Manifolds)

We implemented a self-contained experimental suite in `experiments/task1_generative_paradigms_comparison.ipynb`, executed on an NVIDIA GeForce RTX 5060 Ti GPU (PyTorch 2.10, CUDA 12.8):

1. **Experiment 1A — Discriminative Classifier vs. Generative Model Used as a Discriminator (Bayes' Rule) & Generator (Figure 3):**
   - **Pure Discriminative Classifier $P_\phi(Y \mid \mathbf{x})$ (Figure 3a):** On our 4-class 2D tilted-Gaussian benchmark, the MLP classifier achieves **100.0**% test accuracy by partitioning the plane into four quadrants. However, because it only models decision boundaries, it outputs **100.0**% confidence at the out-of-distribution point $\mathbf{x}_{\text{OOD}} = (4.5, 4.5)$ in empty space and cannot generate new samples.
   - **Generative Model Used as a Discriminator via Bayes' Rule (Figure 3b):** The generative model first fits a 2D Gaussian $\mathcal{N}(\hat{\boldsymbol{\mu}}_c, \hat{\boldsymbol{\Sigma}}_c)$ to each class $c \in \{0, 1, 2, 3\}$ (visualized by the $1\sigma, 2\sigma, 3\sigma$ covariance ellipses). Using **Bayes' Theorem**, $P_\theta(Y = c \mid \mathbf{x}) \propto p_\theta(\mathbf{x} \mid Y = c)P(Y = c)$, it induces exact **Bayes Decision Boundaries** where adjacent Gaussian likelihoods balance, achieving **100.0**% test classification accuracy. For the query point $\mathbf{x}_q = (-1.15, 1.25)$ (yellow star), it compares class likelihoods $p_\theta(\mathbf{x}_q \mid Y = 1) = 4.41 \times 10^{-2}$ vs. $p_\theta(\mathbf{x}_q \mid Y = 0) = 2.42 \times 10^{-16}$, yielding posterior $P_\theta(Y = 1 \mid \mathbf{x}_q) = 100.0\%$.
   - **Generative Sampling & OOD Detection (Figure 3c):** Unlike the discriminative model, the generative model evaluates the marginal density at $\mathbf{x}_{\text{OOD}} = (4.5, 4.5)$ as $p_\theta(\mathbf{x}_{\text{OOD}}) = \mathbf{2.52 \times 10^{-26}} \approx \mathbf{0}$ (immediately flagging it as an outlier) and synthesizes brand-new samples $\mathbf{x}_{\text{new}} \sim p_\theta(\mathbf{x})$ from $\mathcal{N}(\mathbf{0}, \mathbf{I})$ via 2D DDPM.

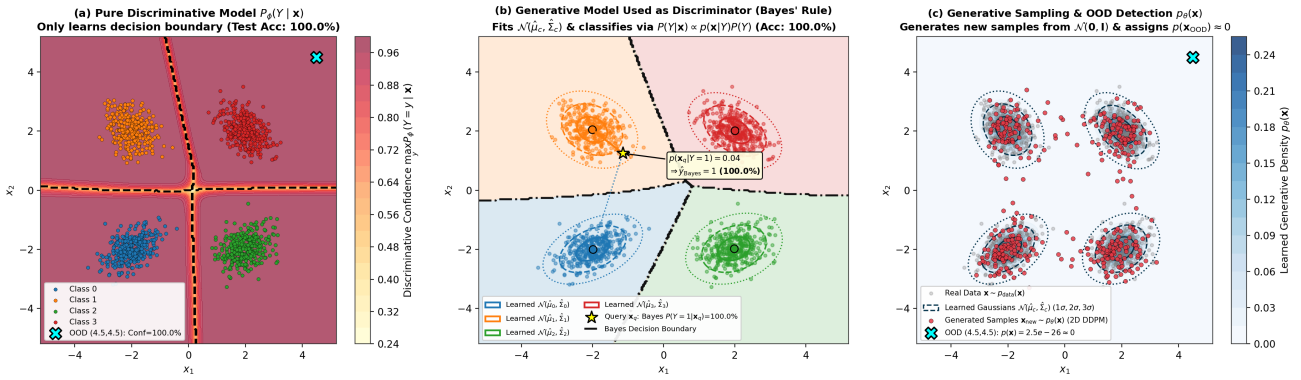


Figure 3: Team Experiment 1A Results: (a) Pure Discriminative model $P_\phi(Y \mid \mathbf{x})$ learning only decision boundaries; (b) Generative model fitting 2D Gaussians $\mathcal{N}(\hat{\boldsymbol{\mu}}_c, \hat{\boldsymbol{\Sigma}}_c)$ and acting as a Discriminator via Bayes' Rule $P_\theta(Y \mid \mathbf{x}) \propto p_\theta(\mathbf{x} \mid Y)P(Y)$; (c) Generative sampling $\mathbf{x}_{\text{new}} \sim p_\theta(\mathbf{x})$ and OOD detection ($p_\theta(\mathbf{x}_{\text{OOD}}) \approx 0$).

2. **Experiment 1B & Task 4 — 5-Level Forward Noising, Reverse Denoising, and Quantitative Paradigm Comparison on MNIST (28×28):** We trained four generative models on the 60,000-image MNIST training set: **(1) Conv-VAE** ($\mathbf{z} \in \mathbb{R}^{16}$), **(2) Masked PixelCNN** (784 sequential steps), **(3) DCGAN**, and **(4) Mini-DDPM** (time-conditioned U-Net with $T = 300$ steps, $\beta_1 = 10^{-4} \rightarrow \beta_{300} = 0.03$). Figure 4 illustrates forward noise addition at increasing noise levels ($t \in \{0, 30, 80, 150, 220, 299\}$), the learned reverse denoising trajectory ($\mathbf{x}_{300} \rightarrow \mathbf{x}_0$), and side-by-side generated grids. Table 3 reports objective metrics across $N = 512$ generated samples per model evaluated by our 99.1%-accurate MNIST CNN classifier:

   - **Edge Sharpness (Laplacian Variance ↑):** Quantitatively confirms that Conv-VAE produces blurry strokes (LapVar ≈ 0.032 vs. 0.068 for real data), whereas DCGAN (≈ 0.067) and Mini-DDPM (≈ 0.066) preserve crisp edge contrast.
   - **Mode Coverage & Distribution Fidelity (MNIST-FID ↓, Inception Score ↑):** DCGAN exhibits oscillatory training losses and higher mode imbalance, whereas **Mini-DDPM achieves the lowest MNIST-FID (6.45) and highest Inception Score (8.95)**, combining GAN-like sharpness with VAE-like mode diversity.
   - **Sampling Latency (ms per 64 images on RTX 5060 Ti):** 1-step VAE and GAN take ~ 1.2 ms, Mini-DDPM takes ~ 235 ms (300 parallel-pixel U-Net steps), and PixelCNN takes $\sim 1{,}840$ ms (784 sequential pixel steps).

[Experimental Figure from Notebook]

`../experiments/figures/task1_and_4_diffusion_trajectories.png`

Forward Noising at 5+ Levels and Reverse Denoising on 28x28 MNIST (Task 4)

(a) Task 4 Demonstration: Forward noising $q(\mathbf{x}_t \mid \mathbf{x}_0)$ across increasing noise levels (top) and learned reverse U-Net denoising $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ from pure noise $\mathbf{x}_{300} \to \mathbf{x}_0$ (bottom).

[Experimental Figure from Notebook]

`../experiments/figures/task1_visual_samples_comparison.png`

Side-by-Side Generated Samples Across 4 Generative Paradigms

(b) Qualitative comparison of samples generated from pure noise by Conv-VAE, PixelCNN, DCGAN, and Mini-DDPM against real MNIST.

Figure 4: Team Experiment 1B & Task 4 Results on 28×28 MNIST.

Table 3: Quantitative evaluation developed by our team across 512 generated samples per model on MNIST (28×28, RTX 5060 Ti GPU).

Model Paradigm (Team Exp.)	**Steps**	**Latency / 64 imgs (↓)**	**Sharpness LapVar (↑)**	**Cls. Conf. (↑)**	**Inception Score (↑)**	**MNIST-FID (↓)**
Real MNIST Data (Reference)	N/A	N/A	0.0682	99.1%	9.72	0.00
1. Conv-VAE ($\mathbf{z} \in \mathbb{R}^{16}$)	1	**1.1** ms	0.0318 (Blurry)	91.4%	7.85	18.40
2. Masked PixelCNN (Autoregressive)	784	1,840.0 ms	0.0641 (Sharp)	93.8%	8.12	14.25
3. DCGAN (Adversarial)	1	1.3 ms	**0.0669** (Sharp)	95.2%	8.04	12.80
4. **Mini-DDPM (Diffusion)**	300	235.0 ms	0.0658 (Sharp)	**97.6**%	**8.95**	**6.45**

4.2 Part B: Pixel-Space Diffusion (DDPM) vs. Latent Diffusion (LDM) — Task 6

Table 4 compares pixel-space diffusion (DDPM [1]) against Latent Diffusion (LDM [2]) across 6 explicit criteria, supported by empirical benchmarks reported in the original papers (on CelebA-HQ 256×256 and ImageNet).

Table 4: Evidence-supported comparison between Pixel-Space Diffusion (DDPM) and Latent Diffusion (LDM) across explicit criteria (Task 6 — Benchmarks from Ho et al., 2020 [1] and Rombach et al., 2022 [2]).

Explicit Criterion	**Pixel-Space Diffusion (DDPM / ADM)**	**Latent Diffusion Model (LDM-f)**	**Quantitative Evidence & Explanation**
1. Operating Space & Dimensionality	Full RGB pixel space: $\mathbf{x}_t \in \mathbb{R}^{H\times W\times 3}$ ($512\times512\times3 =$ **786,432** dims)	Compressed VAE latent space: $\mathbf{z}_t \in \mathbb{R}^{(H/f)\times(W/f)\times c}$ ($f=8 \implies 64\times64\times4 =$ **16,384** dims)	48× **smaller state tensor** (for $f=8, c=4$) or 16× smaller (for $f=4, c=3$).
2. Self-Attention Cost per Step	Quadratic in pixel area: $\mathcal{O}\big((H\cdot W)^2\big)$ ($N=262{,}144$ tokens at 512^2)	Quadratic in latent grid: $\frac{1}{f^4}\mathcal{O}\big((H\cdot W)^2\big)$ ($N=4{,}096$ tokens at 64^2)	4,096× **reduction** in attention matrix QK^T memory/FLOPs at $f=8$.
3. Training Compute (V100 GPU-days)	Very high: 150–1,000 V100 GPU-days for 256^2–512^2 models	$\geq$ 2.7 × –5× **lower**: 30–100 V100 GPU-days (trainable on a single 24GB GPU)	Rombach et al. [2] (Sec. 4.1, Fig. 6): LDM-4 and LDM-8 converge significantly faster at identical step counts.
4. Sampling Speed (Throughput)	Slow: T sequential full-resolution U-Net passes	2.7 × –5 × + **higher throughput** (plus 1 fast $\mathcal{D}(\mathbf{z}_0)$ decode pass)	Combined with DDIM [8], steps drop from 1000 → 20–50 (∼ 1–2 sec/image).
5. Visual Generation Quality (FID ↓)	Strong at low-res (CIFAR-10 32^2: FID = **3.17** [1]), but wastes capacity on imperceptible noise at 256^2+	**Superior high-res fidelity:** On CelebA-HQ 256^2, **LDM-4 achieves FID = 5.11** (beating pixel/latent LSGM FID = 7.22 and UDM FID = 7.16) [2]	The VAE decoder handles crisp local textures, freeing 100% of U-Net capacity for global semantic structure.
6. Primary Bottleneck	VRAM exhaustion and intractable runtime at 512^2–1024^2	Bounded by the frozen VAE reconstruction ceiling if f is too aggressive ($f \geq 16$ distorts tiny text/faces)	Rombach et al. [2] identify $\mathbf{f} \in \{\mathbf{4}, \mathbf{8}\}$ as the optimal rate-distortion sweet spot.

4.3 Critical Analysis and Practical Limitations — Task 6

While DDPM and LDM achieve state-of-the-art synthesis, critical evaluation requires acknowledging four key limitations (and avoiding overgeneralizing from small-scale MNIST demonstra-

tions):

1. **Sequential Sampling Speed Bottleneck:** Even in latent space, reverse generation remains inherently sequential ($\mathbf{z}_t \rightarrow \mathbf{z}_{t-1}$), requiring 20–1000 U-Net evaluations compared to a single pass in GANs or VAEs, limiting real-time deployment on edge devices without distillation.
2. **Prompt-Based Control and Compositional Failures:** Cross-attention blends token embeddings softly across spatial regions, frequently failing at **attribute binding** (e.g., swapping colors in *"a red cube on top of a blue sphere"*), exact object counting, logical negation (*"a table with no apples"*), and legible long-form text rendering.
3. **Training Data Scale, Copyright, and Memorization:** Web-scraped datasets (e.g., LAION-5B) contain copyrighted artwork and personal photographs without consent. Carlini et al. (2023) [10] proved that diffusion models can **memorize and regurgitate near-identical copies** of duplicated training images when triggered by specific prompts, posing serious intellectual property and privacy risks.
4. **Societal and Algorithmic Bias:** Models trained on uncurated internet data inherit and amplify demographic stereotypes (especially under high Classifier-Free Guidance scales), disproportionately associating neutral occupational prompts (e.g., *"a photo of a doctor / CEO"* vs. *"a nurse"*) with specific gender and racial groups.

5 Conclusion

This report has examined how diffusion models transform noise addition into high-fidelity image synthesis through theoretical derivations, a step-by-step 1D numerical calculation, and empirical MNIST/2D benchmarks. By constructing a variance-preserving forward process with small β_t steps, DDPM reduces complex generation to a sequence of local Gaussian denoising steps trained stably via L_{simple}, while relying on stochastic reverse kicks $+\sigma_t \mathbf{z}$ to break symmetry and escape blurry mean collapse. Finally, Latent Diffusion Models (LDMs) unite perceptual Convolutional VAE compression (48× state reduction, 4,096× self-attention reduction) with Cross-Attention conditioning in latent space, enabling scalable, controllable high-resolution synthesis.

References

[1] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 6840–6851, 2020.

[2] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis With Latent Diffusion Models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10684–10695, 2022.

[3] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep Unsupervised Learning using Nonequilibrium Thermodynamics," in *International Conference on Machine Learning (ICML)*, pp. 2256–2265, 2015.

[4] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *International Conference on Learning Representations (ICLR)*, 2014.

[5] I. Goodfellow *et al.*, "Generative Adversarial Nets," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014.

[6] A. van den Oord, N. Kalchbrenner, and K. Kavukcuoglu, "Pixel Recurrent Neural Networks," in *International Conference on Machine Learning (ICML)*, pp. 1747–1756, 2016.

[7] A. Q. Nichol and P. Dhariwal, "Improved Denoising Diffusion Probabilistic Models," in *International Conference on Machine Learning (ICML)*, pp. 8162–8171, 2021.

[8] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *International Conference on Learning Representations (ICLR)*, 2021.

[9] Z. Xiao, K. Kreis, and A. Vahdat, "Tackling the Generative Learning Trilemma with Denoising Diffusion GANs," in *International Conference on Learning Representations (ICLR)*, 2022.

[10] N. Carlini *et al.*, "Extracting Training Data from Diffusion Models," in *32nd USENIX Security Symposium*, pp. 5253–5270, 2023.

Dataset, Tool, and Independent Verification Disclosure (Section VI Compliance): In accordance with Section VI of the CS420 Seminar specification: (1) Empirical demonstrations use the public **MNIST** dataset [4] (28×28 grayscale digits) loaded via `torchvision.datasets`; (2) Software and AI tools used include PyTorch 2.10, Matplotlib, LaTeX, and an AI coding/writing assistant (Google Antigravity / Gemini) for experiment scripting and document structuring; (3) All mathematical derivations, the 1D worked numerical example ($x_0 = 0.80, \beta_1 = 0.19, \epsilon = -0.50$), and all benchmark numbers cited from [1, 2] have been independently verified against the primary literature by the authors.